



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2025/2026

SECP 3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 02

Tutorial 4 - Gen AI

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NO
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
WU CHEN XI	X25EC3007

What is AI-assisted data engineering

AI-assisted data engineering uses Large Language Models (LLMs) to automate repeated and administrative tasks in the data operations pipeline. Data engineers do not depend on AI to design architecture in this workflow. They are using AI as a co-pilot for commenting code, explaining pipelines, enriching metadata, and visualizing structures. As creating a detailed documentation of the systems is a complex task, especially when on an old system, and more so because the pipelines are already running, it is recommended to standardise the selections creating a detailed documentation to avoid reverse engineering.

What is AI agent used in the tutorial?

The tutorial demonstrates the use of Nexla's built-in AI Agent to assist in the development and documentation of data pipelines. Rather than manually configuring every component, engineers interact with the AI Agent through Nexla Express to accelerate pipeline creation, transformation design, and documentation generation.

The AI Agent operates directly within the Nexla platform and understands the pipeline context from the current workspace. Its primary capabilities include:

- **Pipeline generation from natural language prompts:** Users can describe the desired data flow, and the AI Agent automatically generates the corresponding ETL pipeline components.
- **Pipeline explanation and documentation:** The AI Agent summarizes data flow logic, transformations, source-to-target mappings, and pipeline behavior in a human-readable format.
- **Code and transformation assistance:** It helps generate and explain transformation logic, expressions, and configuration settings used throughout the pipeline.
- **Metadata enrichment and synchronization:** Documentation and metadata are automatically updated as pipeline configurations change, reducing manual maintenance effort.

By using the AI Agent as a development co-pilot, data engineers can focus on pipeline design and business requirements while reducing the time spent on repetitive configuration and documentation tasks.

Steps to do prompt and coding

This section will document the end-to-end process of the development of the ETL (Extract, Transform, Load) pipeline, starting from the environment configuration to data verification in Snowflake.

Step 1: Submit Initial Prompt in Nexla Express

Start a new conversation at express.dev, where the following Table 1 will demonstrate the pipeline creation.

Table 1: Prompt Used and Response

Prompt Used	Response
“I want to create a pipeline from the weather api at WeatherAPI.com to my Snowflake table WEATHER_DATA. It should run every 2 minutes ”	Nexla Express understood the intent and began loading connector instructions for both WeatherAPI and Snowflake simultaneously. It confirmed the 2-minute schedule and started gathering credential information.

Step 2: Create [WeatherAPI.com](https://weatherapi.com/) Credential

In order to complete the development of the pipeline, Nexla Express will collect the necessary information for the WeatherAPI.com REST credential, where the configuration authentication input is as shown in Figure 1.

Express
Perfect! Now I'll create a form to collect the necessary information for the WeatherAPI.com REST credential. WeatherAPI.com uses API Key authentication. Let me gather the required inputs:

Preparing Form... Completed

Create WeatherAPI.com Credential

Configure authentication for WeatherAPI.com

Credential Information

* Credential Name
WeatherAPI.com Credential

API Authentication

* API Key
.....

× Cancel ✓ Create Credential

Figure 1: Create [WeatherAPI.com](https://weatherapi.com/) Credential

Step 3: Create Snowflake Credential

Since there is no record of the existing Snowflake credential, Nexla Express will collect the necessary information on Snowflake connection as shown in Figure 2. Hence, the platform will automatically create a Nexset once the first data is ingested where it will happen at the next 2 minutes interval.

Create Snowflake Credential

Configure authentication for Snowflake

Credential Information

* Credential Name

Snowflake Credential

Connection Configuration

* Host

sc30557.southeast-asia.azure.snowflakecomputing.com

* Port

443

* Database Name

WEATHER_DB

* Warehouse

COMPUTE_WH

* Schema Name

PUBLIC

Authentication

* Authentication Type

Basic Authentication (Username & Password)

* Username

nuralsyah04

* Password

F5a6f0db55*#04

Advanced Settings (Optional)

Role

Leave blank for default role

Query Tag

Optional query tag for observability

Cancel Create Credential

Figure 2: Create Snowflake Credentials

Step 4: Create Snowflake Sink - WEATHER_DATA Table

The source of the data is active and ready to fetch data at the next 2 minutes interval, a Nexset will be automatically created once the data flows through. The sink configuration information is collected in order to continue to be set up in order to complete the pipeline as shown in Figure 3.

Create Snowflake Sink - WEATHER_DATA Table

Configure how weather data will be written to Snowflake

Destination Information

* Destination Name

Snowflake Weather Data Sink

Table Configuration

* Table Name

WEATHER_DATA

* Create Table If Not Exists

Yes - Create table if it doesn't exist

* Insert Mode

INSERT - Append new records

Mapping Configuration

* Mapping Mode

Manual - Define custom attribute to column mapping

× Cancel

✓ Create Sink

Figure 3: Create Snowflake Sink

Step 5: Wait For Nexset Creation and Verify Source

Once the credentials have been validated, Nexla Express will create the SOURCE node and wait for it to run at least once in order to create the Nexset (schema representation of the data). With a scheduled interval of 2 minutes, the pipeline will run using CRON JOB. The node created is shown as in Table 2 with Figure 4 to show the structure of the pipeline.

Table 2: Node details

Node	ID	Status
WeatherAPI Weather Data (Source)	123761	ACTIVE
WeatherAPI Weather Data (Nexset)	428841	ACTIVE

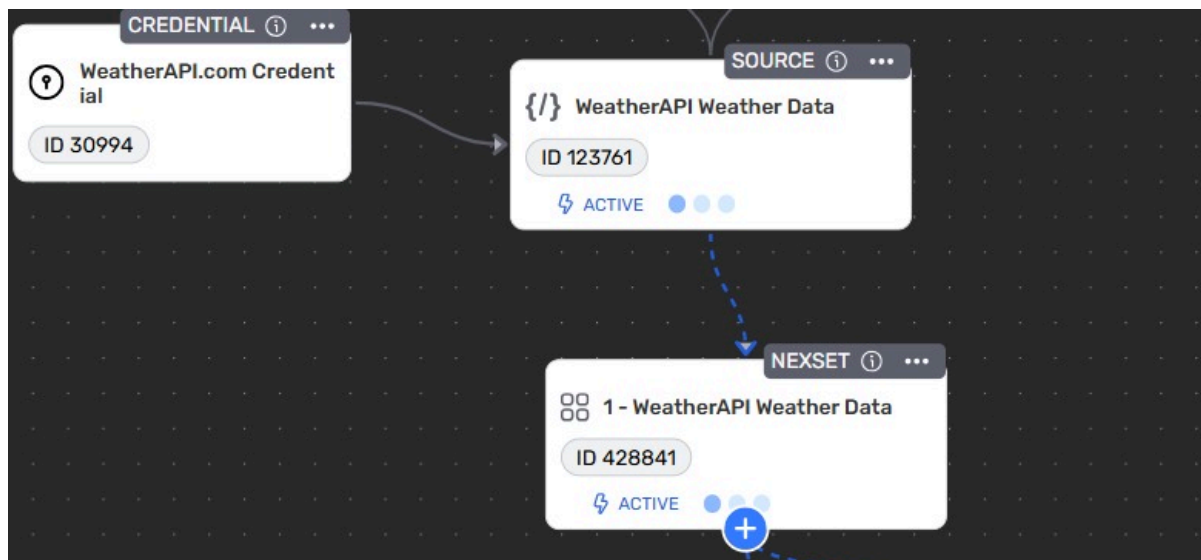


Figure 4: Node Structure

Step 6: Add Fahrenheit Transformation

After the Nexset was ready, the following prompt was submitted to Nexla Express to generate the temperature transformation. Table 3 showing the prompt used and what the AI acts to do the transformation.

Table 3: Prompt Used and Response

Prompt Used	Response
"Add a transformation that converts temperature from Celsius to Fahrenheit using $(C \times 9/5) + 32$ and save it as temp_f column. Place temp_f at root level. Set temp_f to null if current.temp_c is missing."	Nexla Express created a new NEXSET tagged #NexlaAI named 'Weather Data with Fahrenheit Temperature' (ID 428842). The AI generated the conversion formula automatically and handled null cases as specified.

The AI generated transformation logic applied:

- Input field: current_temp_c
- Formula: $\text{temp_f} = (\text{current_temp_c} \times 9/5) + 32$
- Output field: temp_f
- Null handling: If current_temp_c is null, temp_f is set to null

After prompting, Figure 5 shows the node Nexset the transformation of temperature. The connection is also with the weather data so it can transform previous temperature in Celsius to Fahrenheit.

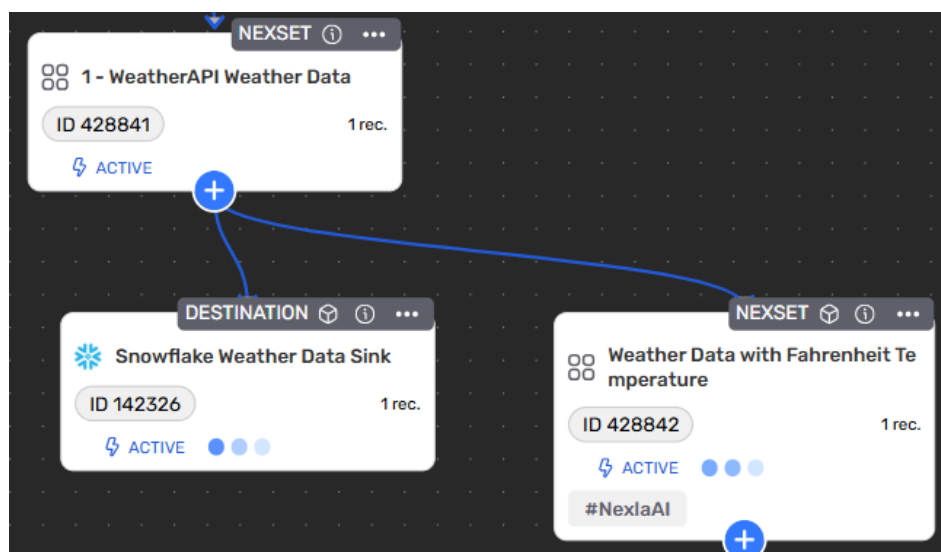


Figure 5: Connection between Weather Data with Fahrenheit and Weather API Data

Step 7: Configure Snowflake Destination Sinks

The destination sinks were created for the transformed data which is the temperature Fahrenheit. Figure 6 shows the new node destination of the data sink was created after completing the transformation.

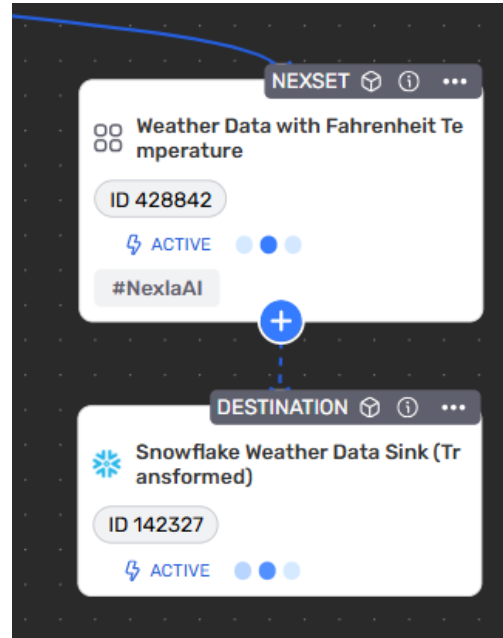


Figure 6: Updated Snowflake Weather Data Sink

Step 8: Complete ETL Pipeline

The pipeline completed after all of the prompts were done. Figure 7 shows the completed ETL pipeline by extracting data from Weather API, transforming the temperature from Celsius to Fahrenheit, and loading it into the Snowflake database.

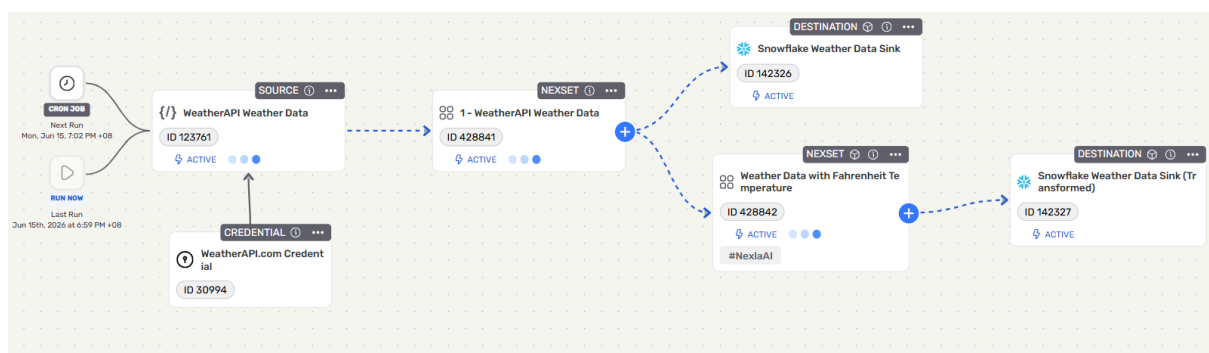


Figure 7: Completed ETL Pipeline

Step 9: Run and Verify Pipeline

The pipeline was triggered using the RUN NOW button and subsequently verified in Snowflake. Figure 8 shows sample data loaded into Snowflake by running SQL query, `SELECT *`

FROM WEATHER_DATA;

	location_name	location_region	location_country	location_lat	location_lon	location...	last_updated	temp_c	temp_f	condition_text	wind_mph	
	Johor Ba... 100.0%	Johor 100.0%	Malaysia 100.0%	1.4667 1.4667	103.75 103.75	2026-06-1... 2.9% 2026-06-1... 2.9% +33 more	2026-06-1... 11.4% 2026-06-1... 11.4% +12 more	29.1 32.4	84.4 90.3	Overcast 54.3% Partly cl... 45.7%	6.7 8.7	10.8
1	Johor Bahru	Johor	Malaysia	1.4667	103.75	2026-06-15 19:09	2026-06-15 19:00	29.1	84.4	Overcast	6.7	
2	Johor Bahru	Johor	Malaysia	1.4667	103.75	2026-06-15 19:02	2026-06-15 19:00	29.1	84.4	Overcast	6.7	
3	Johor Bahru	Johor	Malaysia	1.4667	103.75	2026-06-15 18:50	2026-06-15 18:30	29.4	null	Overcast	7.2	
4	Johor Bahru	Johor	Malaysia	1.4667	103.75	2026-06-15 18:44	2026-06-15 18:30	29.4	84.9	Overcast	7.2	
5	Johor Bahru	Johor	Malaysia	1.4667	103.75	2026-06-15 18:32	2026-06-15 18:15	31.2	88.2	Overcast	7.2	

Figure 8: Sample Data in Snowflake

Use Case Diagram

The use case diagram in completing this tutorial has identified 2 actors, namely as Data Engineer and AI Agent where the use case is shown in Figure 9.

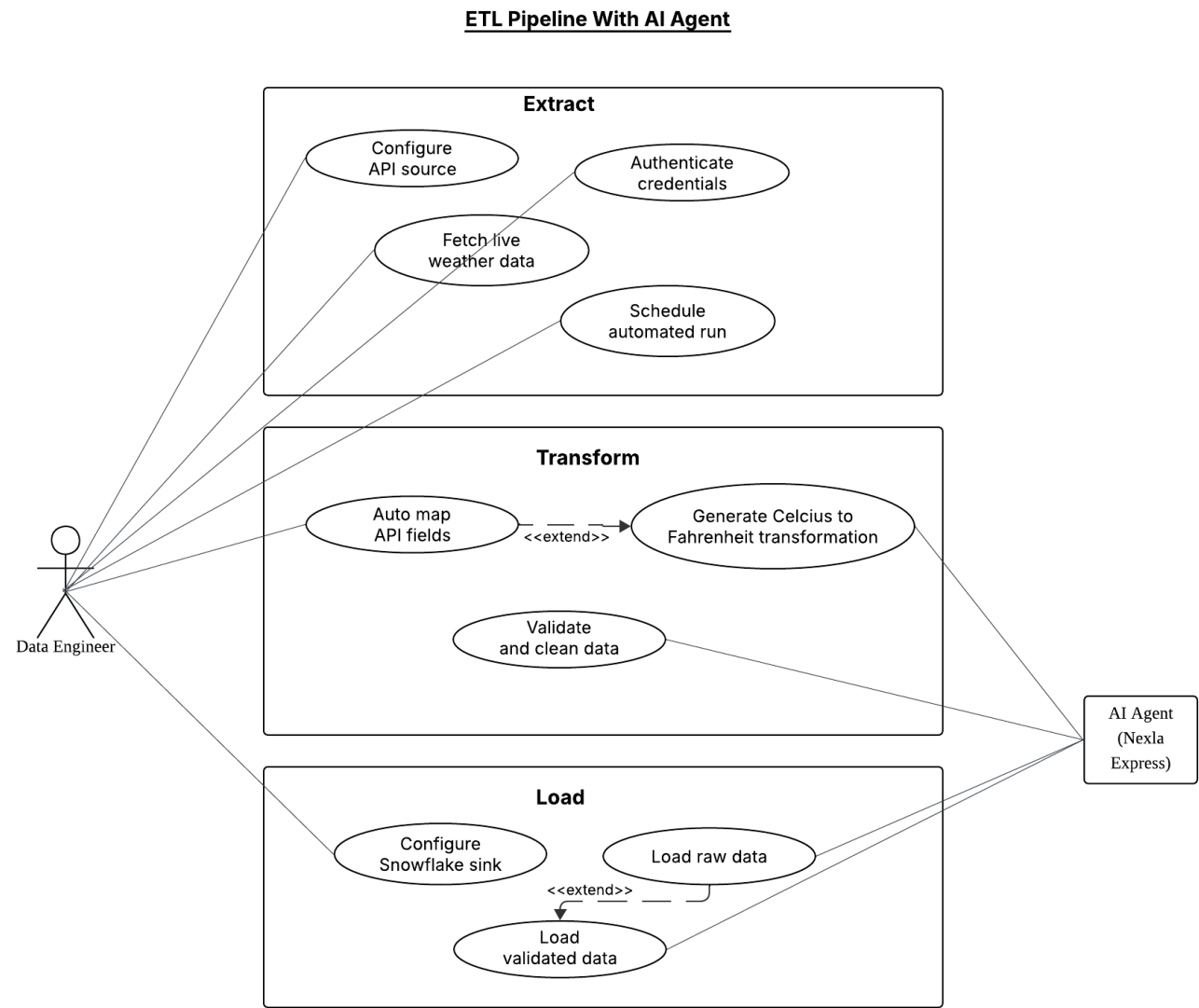


Figure 9: Use Case Diagram

Reflection

1. Nur Firzana Binti Badrus Hisham

This tutorial showed how AI can make building the data pipelines much faster and easier without any coding. Using Nexla Express, a complete ETL pipeline was set up through simple chat prompts, by extracting the live weather data, transforming temperature from celsius to fahrenheit, and loading the results into Snowflake every 2 minutes. The AI handled most of the heavy work automatically, including creating the connectors, mapping data fields, and generating transformation code. However, some limitations were noticed, null values appeared in certain columns due to incorrect field mapping, duplicate records were loaded into Snowflake across multiple runs, and the AI could not connect pipeline paths on its own. These issues highlight that human oversight is still necessary even when using AI tools. In future, adding deduplication logic, improving prompt instructions and expanding the pipeline to cover more cities would make the solution more complete and reliable.

2. Nuraisyah Binti Mohd Zikre

Throughout the completion of this tutorial it provided a firsthand insight into the shift of manual ETL coding to a higher-level system orchestration. By utilizing AI, it has sped up the development process by rapidly structuring the node of the pipeline no matter if the pipeline is automated or not. Besides, while AI tools are powerful accelerators, it also heavily relies on the engineer's domain knowledge to function the automation of the pipeline correctly. Moving forward, this pipeline could further improve the development of a pipeline, by transitioning from a local script to a fully automated cloud scheduler like Apache Airflow or a Snowflake Task. Not to mention the integrated automated data quality checks to ensure the system is completely production-ready.

3. Wu Chen Xi

Through this tutorial, I gained a better understanding of how AI-assisted data engineering can simplify the development of ETL pipelines. Using Nexla Express, the pipeline was created through natural language instructions, allowing data extraction, transformation, and loading tasks to be completed with significantly less manual configuration. The experience demonstrated how AI can accelerate workflow development by generating pipeline

components, configuring data mappings, and assisting with documentation. At the same time, the tutorial highlighted the importance of validating the generated output, as AI-generated configurations may not always match the intended requirements. Small issues such as mapping inconsistencies and data quality concerns still required manual review and correction. Overall, this exercise showed that AI is a valuable productivity tool for data engineers, helping reduce repetitive work while enabling faster prototyping. In the future, the pipeline could be enhanced with stronger validation rules, error-handling mechanisms, and support for additional data sources to improve scalability and reliability.